



**POLITECNICO
DI TORINO**

Dipartimento
di Automatica e Informatica

Code a Priorità e Heap

Gianpiero Cabodi e Paolo Camurati

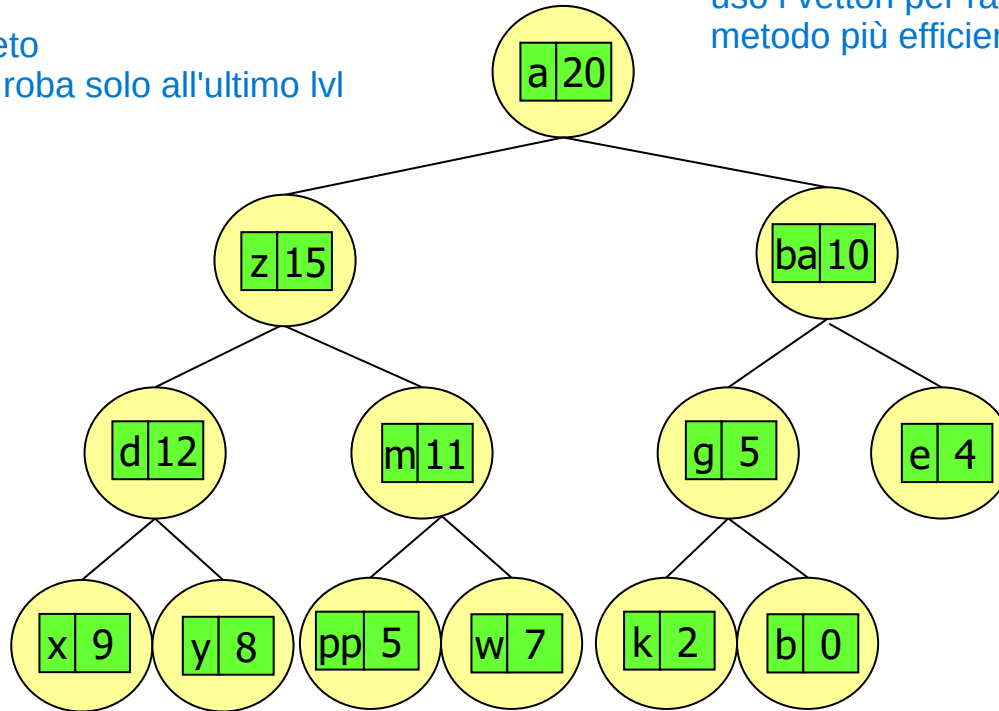
ADT Heap

- Definizione: albero binario con
 - **proprietà strutturale:** quasi completo (tutti i livelli completi, tranne eventualmente l'ultimo, riempito da SX a DX) $\Rightarrow$ quasi bilanciato
 - **proprietà funzionale:** il padre di quel nodo ha un valore più grande (o uguale)
$$\forall i \neq r \quad \text{key}(\text{parent}(i)) \geq \text{key}(i)$$
 - conseguenza: chiave max nella radice
- Implementazione: mediante vettore.

Esempio

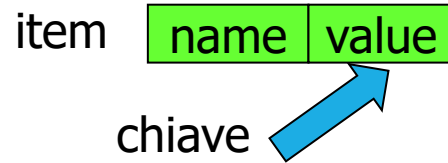
albero completo
può mancare roba solo all'ultimo lvl

heap è albero completo, motivazione per cui
uso i vettori per rappresentarli
metodo più efficiente



Item

- Quasi ADT Item
- Dati:
 - Nome (stringa), valore (intero)
 - Chiave = valore
 - Tipologia 3



ADT di I classe Heap

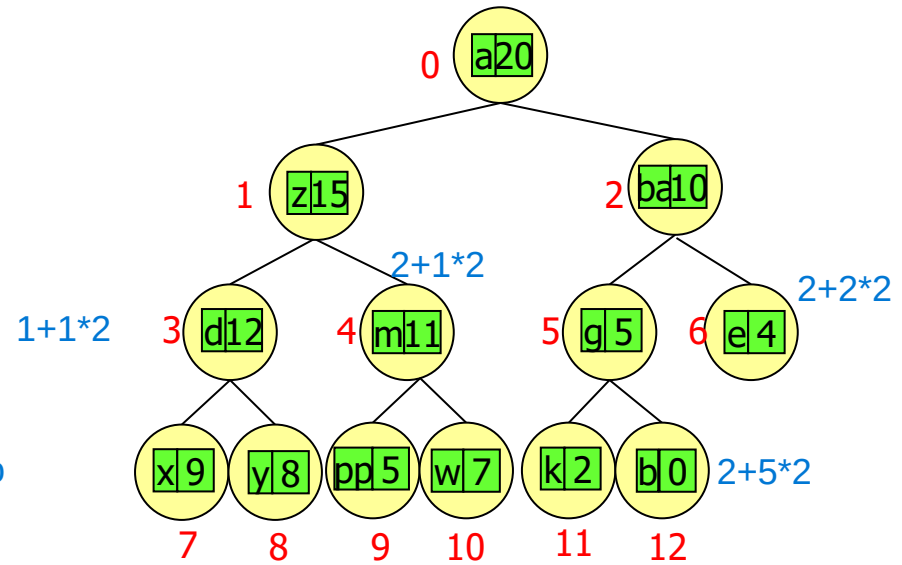
Heap.h

```
typedef struct heap *Heap;  
  
Heap  HEAPinit(int maxN);  
Void  HEAPfree(Heap h);  
void  HEAPfill(Heap h, Item val);  
void  HEAPSORT(Heap h);  
void  HEAPdisplay(Heap h);
```

Implementazione

- Struttura dati: vettore di Item $h \rightarrow A[0..\max N-1]$
- $h \rightarrow \text{heapsize}$: numero di elementi in heap $h \rightarrow A$
- radice in $h \rightarrow A[0]$
- dato $h \rightarrow A[i]$:
 - il figlio SX è $h \rightarrow A[\text{LEFT}(i)]$ dove $\text{LEFT}(i) = 2i+1$
 - il figlio DX è $h \rightarrow A[\text{RIGHT}(i)]$ dove $\text{RIGHT}(i) = 2i+2$
 - il padre è $h \rightarrow A[\text{PARENT}(i)]$ dove $\text{PARENT}(i) = (i-1)/2$ divisione con troncamento

Esempio



maxN = 15 albero completo

h->A

a	z	ba	d	m	g	e	x	y	pp	w	k	b		
20	15	10	12	11	5	4	9	8	5	7	2	0		
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

h->heapsize **13** nodi effettivamente presenti

Heap.c

```
#include <stdio.h>
#include <stdlib.h>
#include "Item.h"
#include "Heap.h"

struct heap { Item *A; int heapsize; };

int LEFT(int i) { return (i*2 + 1); } solo per nascondere
int RIGHT(int i) { return (i*2 + 2); }
int PARENT(int i) { return ((i-1)/2); }

Heap HEAPinit(int maxN) {
    Heap h;
    h = malloc(sizeof(*h));
    h->A = malloc(maxN*sizeof(Item));
    h->heapsize = 0;
    return h;
}
```

indice figlio sx

indice figlio dx

usata per inserire valori,
non necessariamente
il risultato sarà uno heap

```
void HEAPfree(Heap h) {  
    free(h->A);  
    free(h);  
}  
  
void HEAPfill(Heap h, Item item) {  
    int i;  
    i = h->heapsize++;  
    h->A[i] = item;  
    return;  
}  
  
void HEAPdisplay(Heap h) {  
    int i;  
    for (i = 0; i < h->heapsize; i++)  
        ITEMstore(h->A[i]);  
}
```

Funzione HEAPify

aggiusta un heap che non va bene, trasforma in heap qualcosa che sarebbe stato un heap a meno di una e una sola violazione

- Trasforma in heap i , $\text{LEFT}(i)$, $\text{RIGHT}(i)$, dove $\text{LEFT}(i)$ e $\text{RIGHT}(i)$ sono già heap
- assegna ad $A[i]$ il max tra $A[i]$, $A[\text{LEFT}(i)]$ e $A[\text{RIGHT}(i)]$
- se c'è stato scambio $A[i] \leftrightarrow A[\text{LEFT}(i)]$, applica ricorsivamente HEAPify su sottoalbero con radice $\text{LEFT}(i)$
- analogamente per scambio $A[i] \leftrightarrow A[\text{RIGHT}(i)]$.
- Complessità: $T(n) = O(\lg n)$.

ogni volta l'ampiezza di dimezza (sottoalbero sx oppure dx)
ho un singolo sottoproblema (guardo solo uno dei due sottoalberi)
costo di divisione e ricombinazione lineare (in loco)
eq. ricorrenze: $T(n) = T(n/2) + 1$

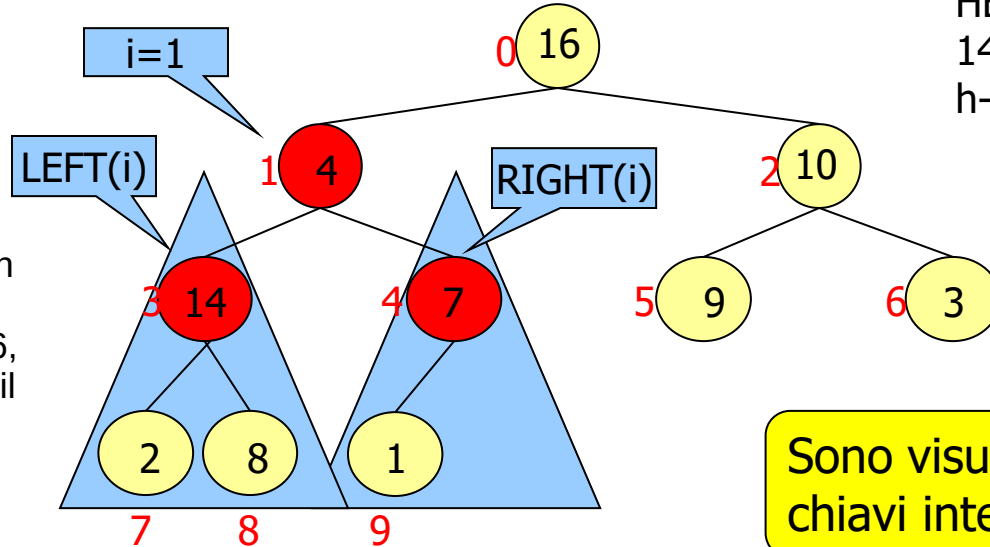
Esempio

va bin solo per terne
che sono in errore

prendo il più grande dei figli e lo
scambio col nodo che rompe le
regole (nodo 1 di valore 4)

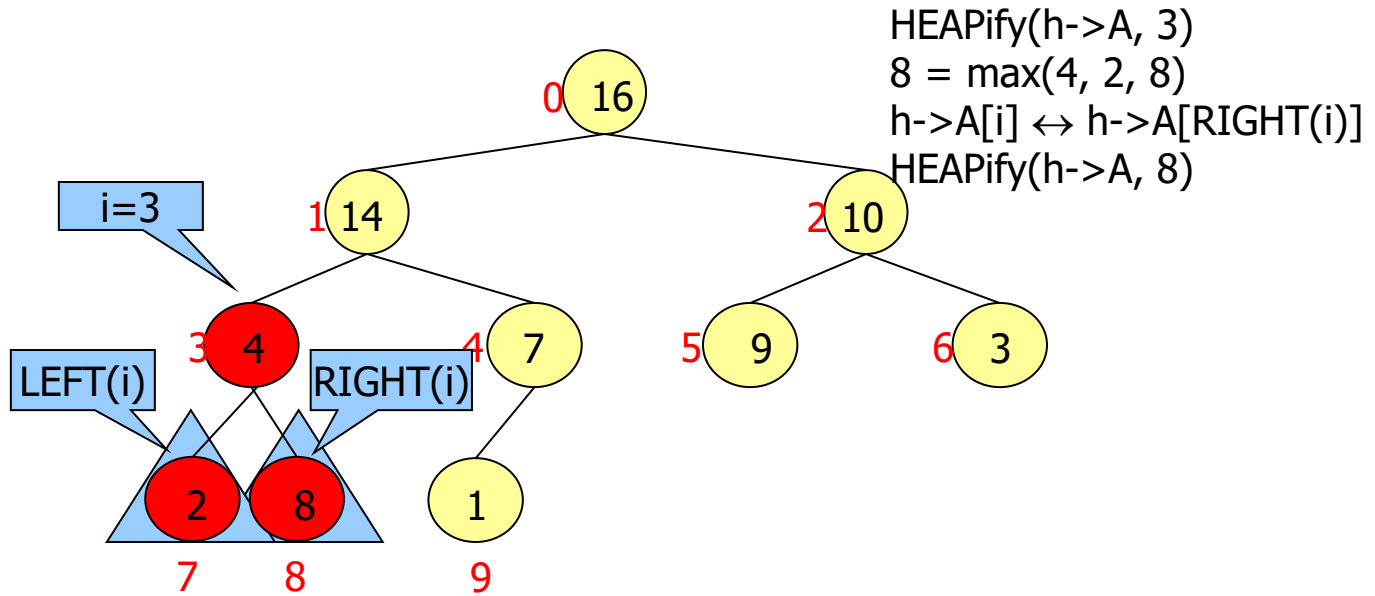
se non va ancora bin scambia di nuovo il
nodo di valore 4 (al nuovo posto) col
maggiore del suo figlio

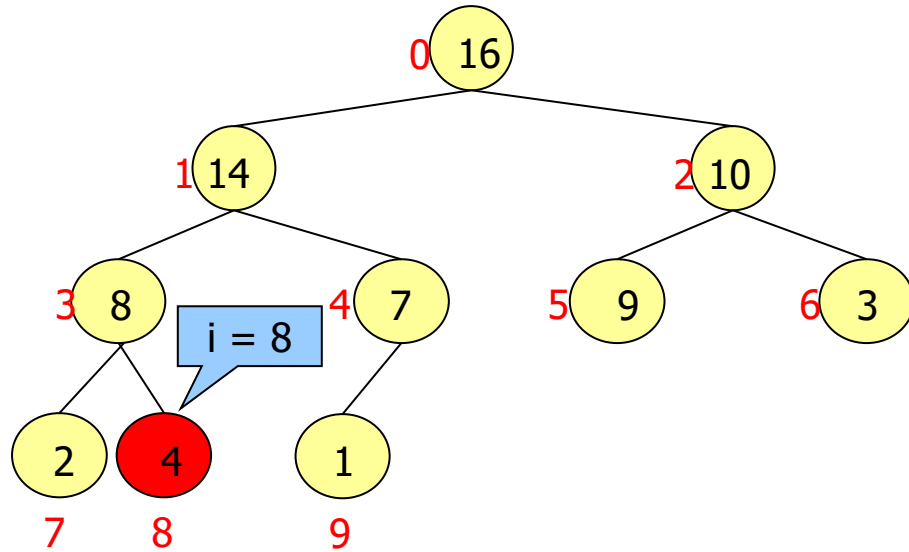
se avessi
avuto in 3 un
valore più
grande di 16,
viola anche il
16?



HEAPIfy(h->A, 1)
 $14 = \max(4, 14, 7)$
 $h \rightarrow A[i] \leftrightarrow h \rightarrow A[\text{LEFT}(i)]$

Sono visualizzate solo le
chiavi intere, non gli item





HEAPIfy(h->A, 8)
foglia
terminazione.

ricerca del
max tra 3
valori

```
void HEAPIfy(Heap h, int i) {  
    int l, r, largest;  
    l = LEFT(i);  
    r = RIGHT(i);  
    if ((l < h->heapsize) &&  
        KEYcmp(KEYget(h->A[l]), KEYget(h->A[i])) == 1)  
        largest = l;  
    else                                largest salva l'indice del massimo  
        largest = i;  
    if ((r < h->heapsize) &&  
        KEYcmp(KEYget(h->A[r]), KEYget(h->A[largest])) == 1)  
        largest = r;  
    if (largest != i) {  
        Swap(h, i, largest);  
        HEAPIfy(h, largest);  scendi e ritenta (ricorsione lineare)  
    }  
}
```

Funzione HEAPbuild

uno qualunque

- Trasforma un albero binario memorizzato in vettore A in uno heap:
 - le foglie sono heap
 - applica HEAPIfy a partire dal padre dell'ultima foglia o coppia di foglie fino alla radice.

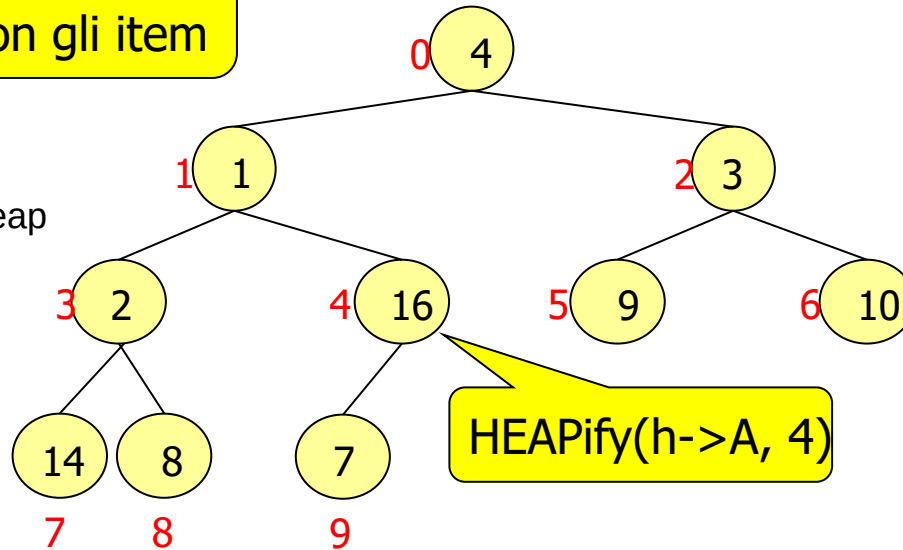
```
void HEAPbuild (Heap h) {  
    int i;  
    for (i=PARENT(h->heapsize-1); i >= 0; i--)  
        HEAPIfy(h, i);  
}
```

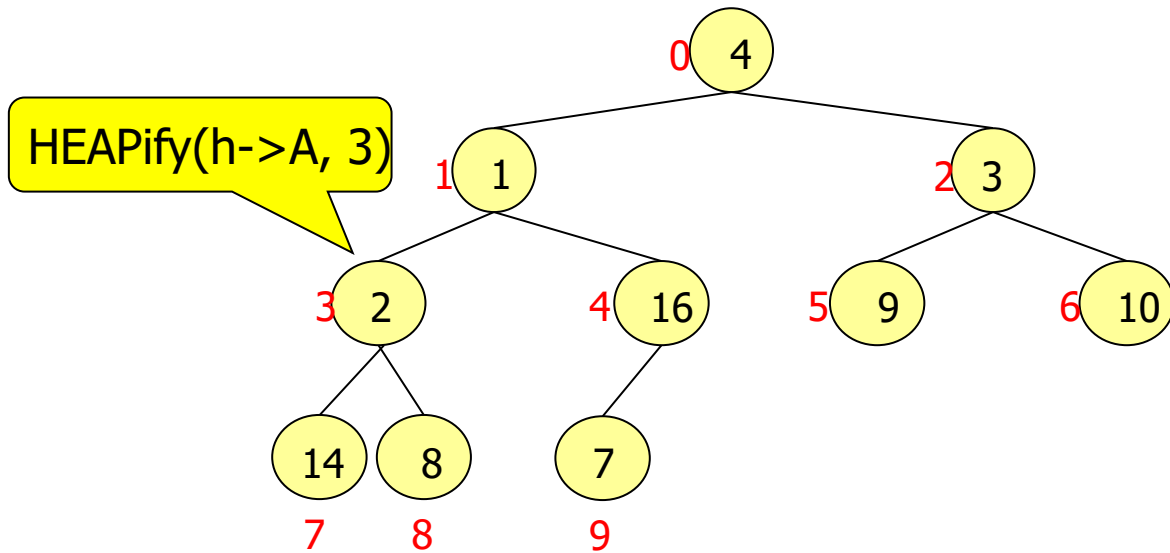
Esempio

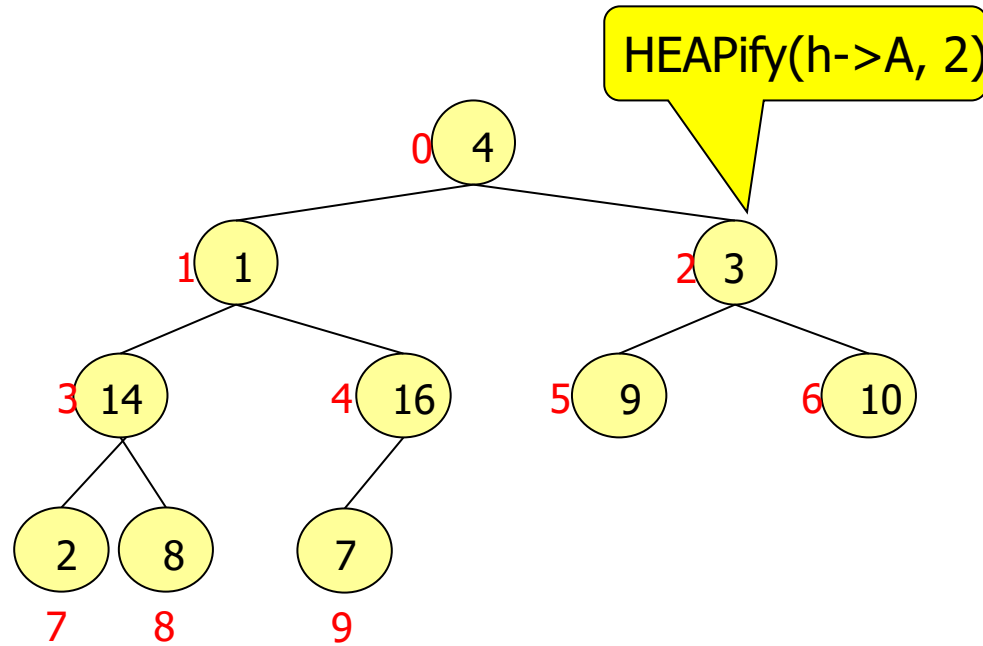
HEAPbuild(h->A)

Sono visualizzate solo le chiavi intere, non gli item

partiamo dal basso
rendi tutti sottoalberi heap

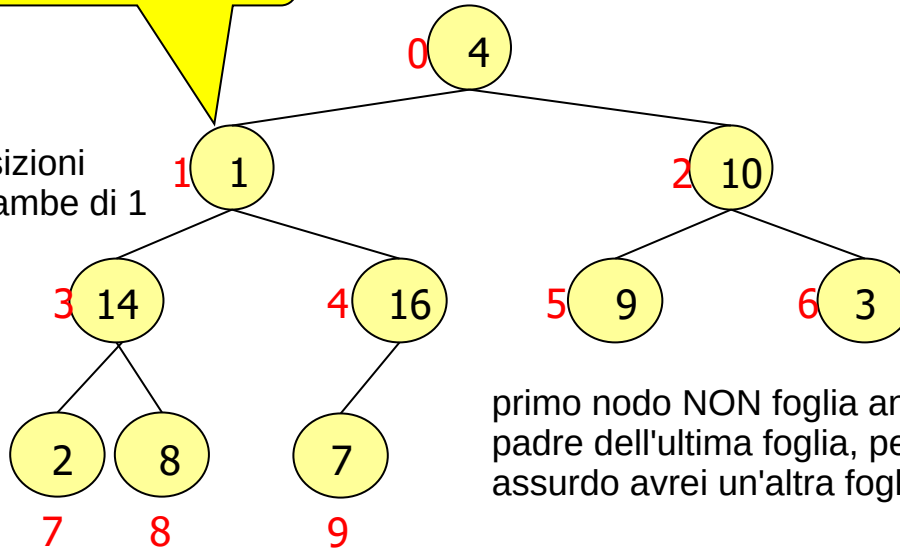






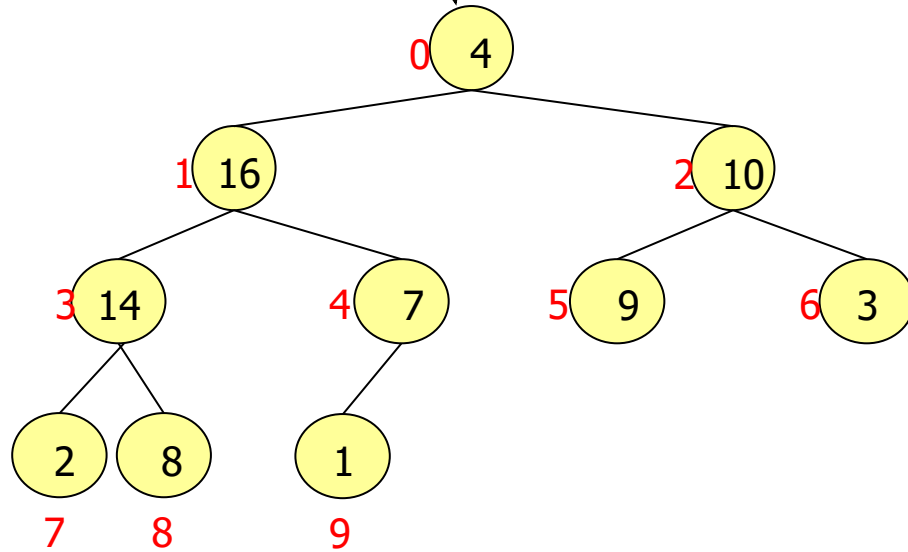
HEAPIfy(h->A, 1)

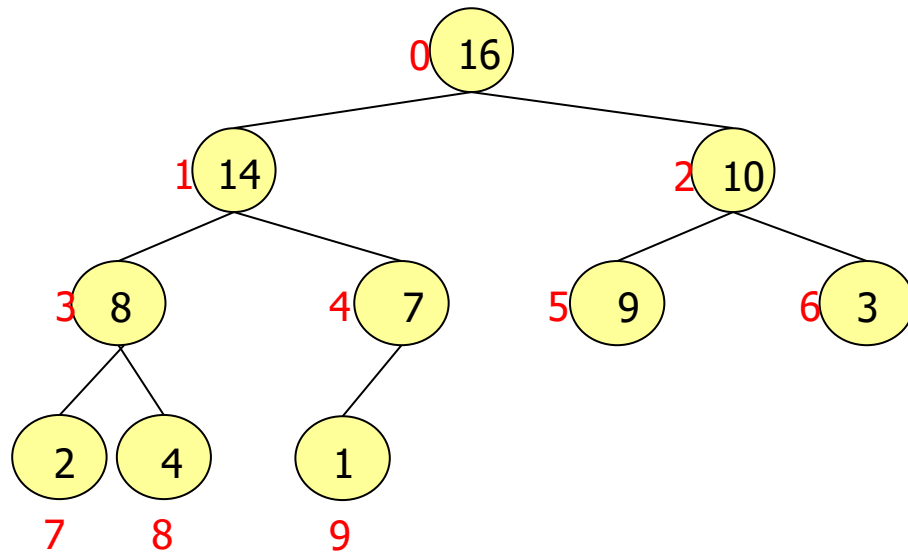
se famo heapify
1 scende di due posizioni
16 e 7 salgono entrambe di 1



primo nodo NON foglia andando a ritroso è anche padre dell'ultima foglia, perchè se non lo fosse per assurdo avrei un'altra foglia?

HEAPIfy(h->A, 0)





■ Analisi di complessità:

- intuitiva ed **imprecisa**: n passi ciascuno di costo $\log n$, quindi $T(n) = O(n \lg n)$
- precisa: **$T(n) = O(n)$** .

Risoluzione per sviluppo (unfolding).

$$T(n) = 2T(n/2) + \log_2(n)$$

$$T(n/2) = 2T(n/4) + \log_2(n/2)$$

$$T(n/4) = 2T(n/8) + \log_2(n/4)$$

2 sottoalberi

Heapify

Sostituendo in T(n):

$$T(n) = \sum_{i=0}^{\log_2 n} 2^i \log_2(n/2^i)$$

$$= \log_2 n \sum_{i=0}^{\log_2 n} 2^i - \sum_{i=0}^{\log_2 n} i 2^i$$

$$= \log_2 n (2n-1) - 2(1-(\log_2 n+1)n+2n\log_2 n)$$

$$= 2n - \log_2 n - 2$$

$$= O(n)$$

$$\sum_{i=0}^k i x^i = x (1-(k+1)x^k + kx^{k+1}) / (1-x)^2$$

alg basato sul confronto

Funzione HEAPsort

- Trasforma il vettore in uno heap mediante HEAPbuild
- Scambia il primo e ultimo elemento
- Riduci la dimensione dello heap di 1
- Ripristina la proprietà di heap tramite heapify le heapify partono tutte dalla radice
- Ripeti fino a esaurimento dello heap.
- Caratteristiche: ho usato n volte heapify
 - complessità: $T(n) = O(n \lg n)$.
 - in loco
 - non stabile

anche se si preferisce il
quick sort in media

Esempio

Sono visualizzate solo le
chiavi intere, non gli item

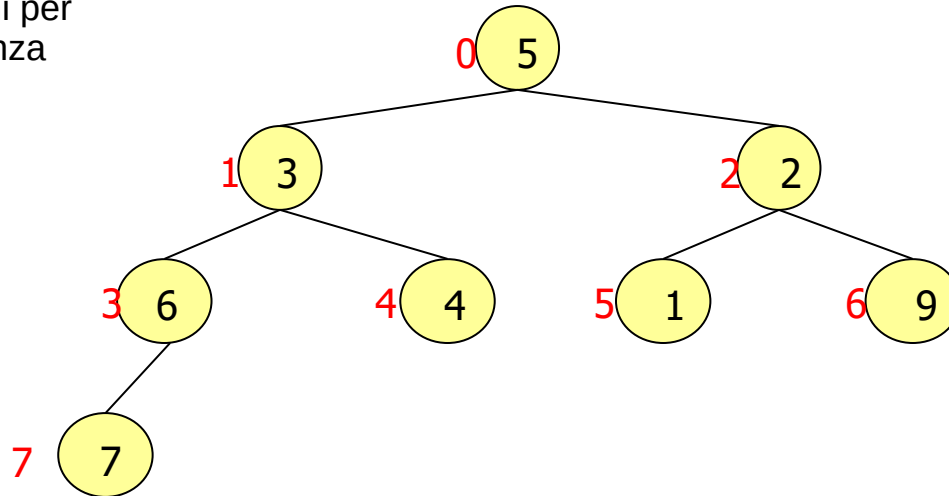
il bello di avere un heap è il massimo in testa

Configurazione iniziale

h->A

5	3	2	6	4	1	9	7
---	---	---	---	---	---	---	---

e avere i massimi per
area di competenza

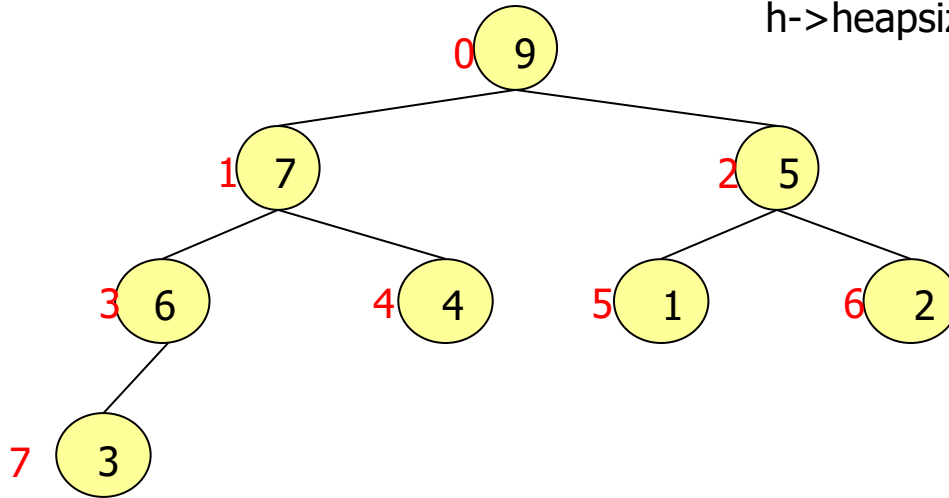


Applico HEAPbuild

h->A

9	7	5	6	4	1	2	3
---	---	---	---	---	---	---	---

h->heapsize = 8



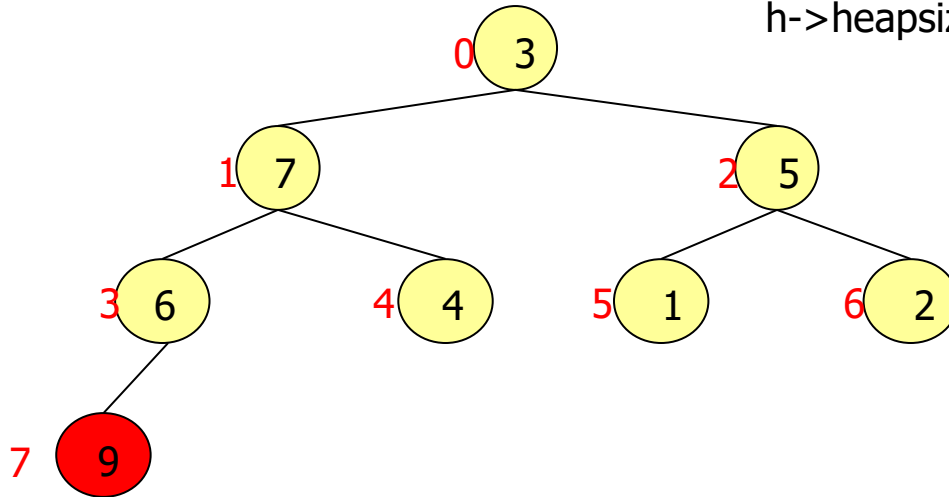
$h \rightarrow A[0] \leftrightarrow h \rightarrow A[h \rightarrow \text{heapsize} - 1]$

$h \rightarrow \text{heapsize}--$

$h \rightarrow A$

3	7	5	6	4	1	2	9
---	---	---	---	---	---	---	---

$h \rightarrow \text{heapsize} = 7$

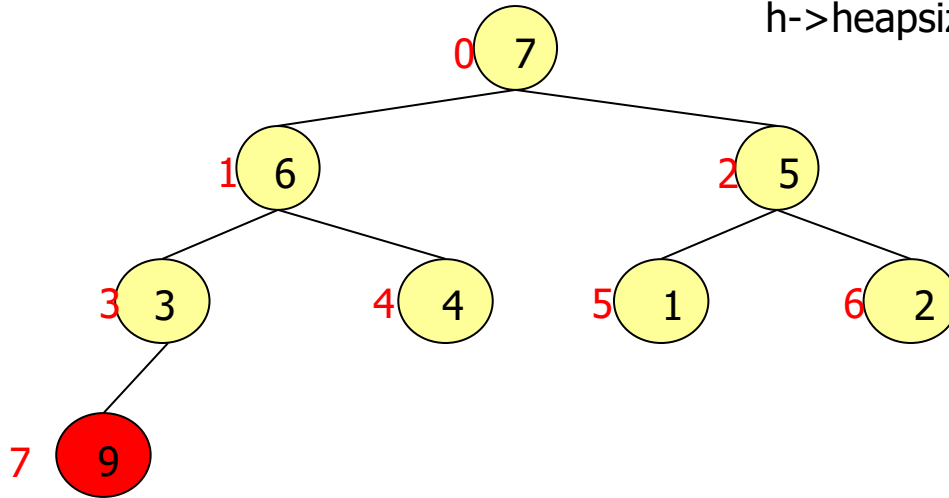


HEAPIfy(h->A, 0)

h->A

7	6	5	3	4	1	2	9
---	---	---	---	---	---	---	---

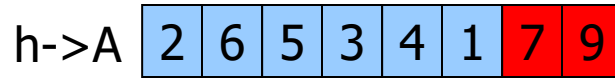
h->heapsize = 7



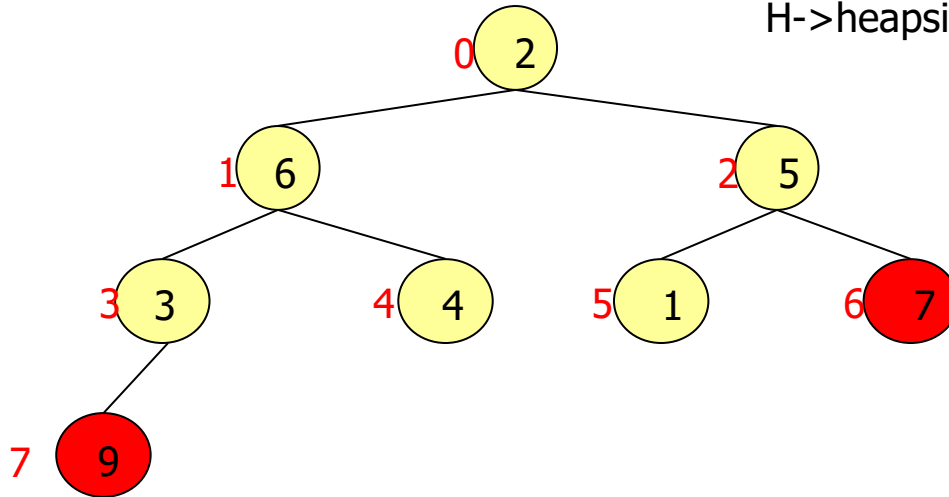
$h \rightarrow A[0] \leftrightarrow h \rightarrow A[h \rightarrow \text{heapsize} - 1]$

$h \rightarrow \text{heapsize}--$

$h \rightarrow A$



$H \rightarrow \text{heapsize} = 6$

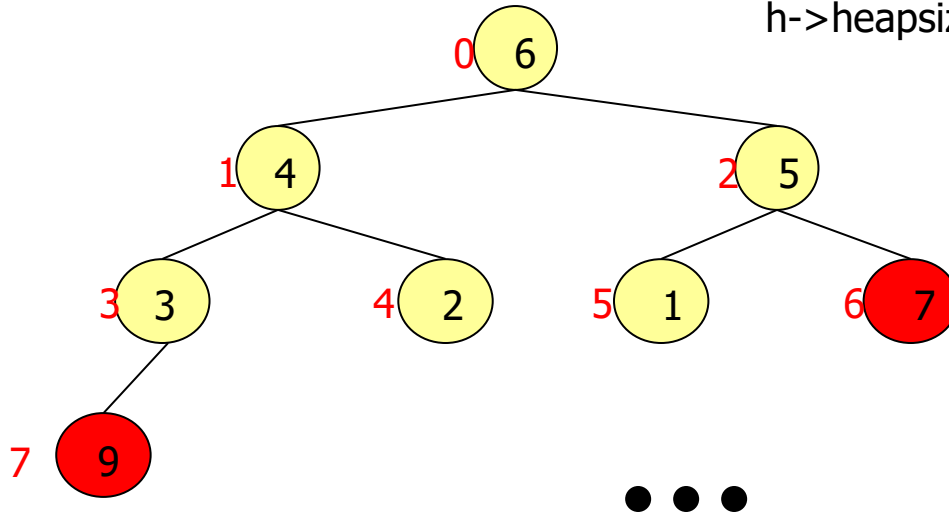


HEAPIfy(h->A, 0)

h->A

6	4	5	3	2	1	7	9
---	---	---	---	---	---	---	---

h->heapsize = 6



ma se i programmatori non lo usano (il heapsort)
non è granchè, a cosa serve realmente un heap?
CODE A PRIORITA'

```
void HEAPsort(Heap h) {  
    int i, j;  
    HEAPbuild(h);  
    j = h->heapsize;  
    for (i = h->heapsize-1; i > 0; i--) {  
        Swap (h,0,i);  
        h->heapsize--;  
        HEAPIfy(h,0);  
    }  
    h->heapsize = j;  
}
```

Coda a priorità

Definizione:

- struttura dati PQ per mantenere un set di elementi di tipo Item, ciascuno dei quali include un campo priorità
- operazioni principali: inserzione, estrazione del massimo, lettura del massimo, cambio di priorità.

ADT di I classe Coda a Priorità

PQ.h

```
typedef struct pqueue *PQ;  
  
PQ      PQinit(int maxN);  
void    PQfree(PQ pq);  
int     PQempty(PQ pq);  
void    PQinsert(PQ pq, Item val);  
Item    PQextractMax(PQ pq);  
Item    PQshowMax(PQ pq);  
void    PQdisplay(PQ pq);  
int     PQsize(PQ pq);  
void    PQchange(PQ pq, Item val);
```

discussione a parte

Implementazione della struttura dati:

- vettore/lista non ordinato
 - vettore/lista ordinato
 - heap di dati/indici.
- } non considerati qui,
cfr Tipi di Dato Astratto

Complessità

	PQinsert	PQshowMax	PQextractMax
Vettore non ordinato	1	N	N
Lista non ordinata	1	N	N
Vettore ordinato	N	1	1
Lista ordinata	N	1	1
Heap di item/indici	logN	1	logN

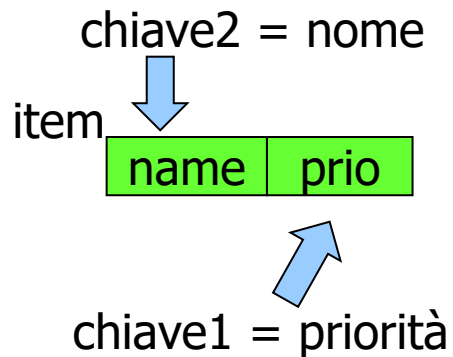
max in coda

max in testa

Cosa contiene l'ADT Coda a priorità?

La soluzione: la **coda a priorità contiene dati** (lo heap che realizza la coda a priorità contiene i dati), l'ADT è una struct con:

1. la coda a priorità: vettore (heap) $pq \rightarrow A$ di dati di tipo Item (quasi ADT, tipologia 3)
2. heapsize: intero.



+ prioritario==numero più grande
un vettore item formato da una stringa e un intero (la sua priorità)

operazione logaritmica per rimpiazzare il massimo (dopo che ho estratto il massimo)

Utente

item  PQinsert(pq, item)

item  PQshowMax(pq)

item  PQextractMax(pq)

ADT I cat. coda a priorità di dati

pq->A

a	q	zz	qa	cd	s	w	c		
81	70	20	48	5	9	19	15		

0 1 2 3 4 5 6 7 8 9

pq->heapsize 8

pq->A

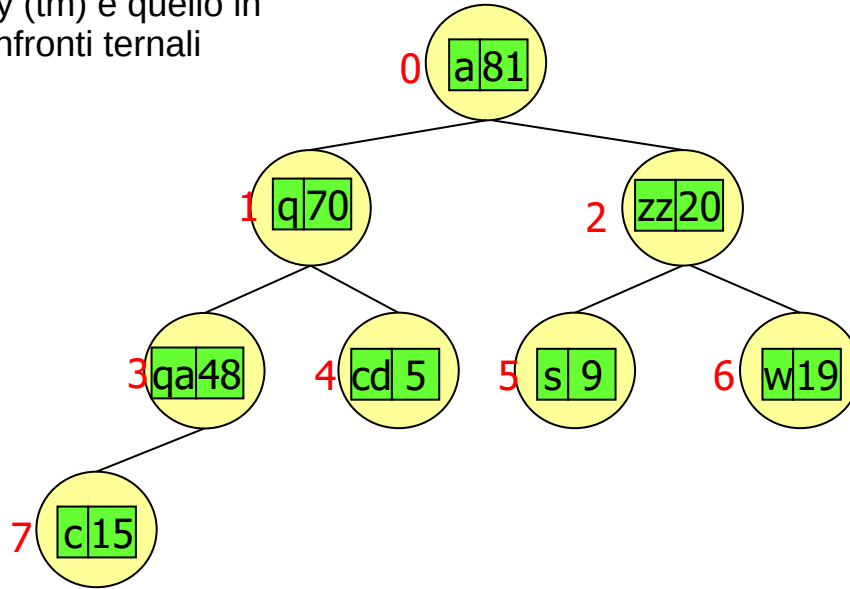
a	q	zz	qa	cd	s	w	c		
81	70	20	48	5	9	19	15		

0 1 2 3 4 5 6 7 8 9

pq->heapsize 8

heapify in salita (heap up) per alcuni

NOI a questo non daremo un nome, il Heapify (tm) è quello in discesa con confronti ternali



ADT di I cat. Coda a priorità

PQ.c

```
#include <stdlib.h>
#include "Item.h"
#include "PQ.h"

struct pqueue { Item *A; int heapsize; };

static int LEFT(int i) { return (i*2 + 1); }
static int RIGHT(int i) { return (i*2 + 2); }
static int PARENT(int i) { return ((i-1)/2); }

PQ PQinit(int maxN){
    PQ pq = malloc(sizeof(*pq));
    pq->A = (Item *)malloc(maxN*sizeof(Item));
    pq->heapsize = 0;
    return pq;
}
```

```
void PQfree(PQ pq){
    free(pq->A);
    free(pq);
}

int PQempty(PQ pq) { return pq->heapsize == 0; }

int PQsize(PQ pq) { return pq->heapsize; }

Item PQshowMax(PQ pq) { return pq->A[0]; }

void PQdisplay(PQ pq) {
    int i;
    for (i = 0; i < pq->heapsize; i++)
        ITEMstore(pq->A[i]);
}
```


Funzione PQinsert

- Aggiunge una foglia all'albero (cresce per livelli da SX a DX, rispettando la proprietà strutturale)
- Risale dal nodo corrente (inizialmente la foglia appena creata) fino al più alla radice. Confronta la chiave del dato contenuto nel padre con la chiave del dato da inserire, facendo scendere il dato del padre nel figlio se la chiave da inserire è maggiore, altrimenti inserisce il dato nel nodo corrente.

Complessità: $T(n) = O(\lg n)$.

ho un heap.
operazione molto simile al heapify ma al contrario

ma NON va chiamata heapify!!
questo è in salita mentre heapify in discesa
questo confronto tra due cose mentre heapify confronto ternario

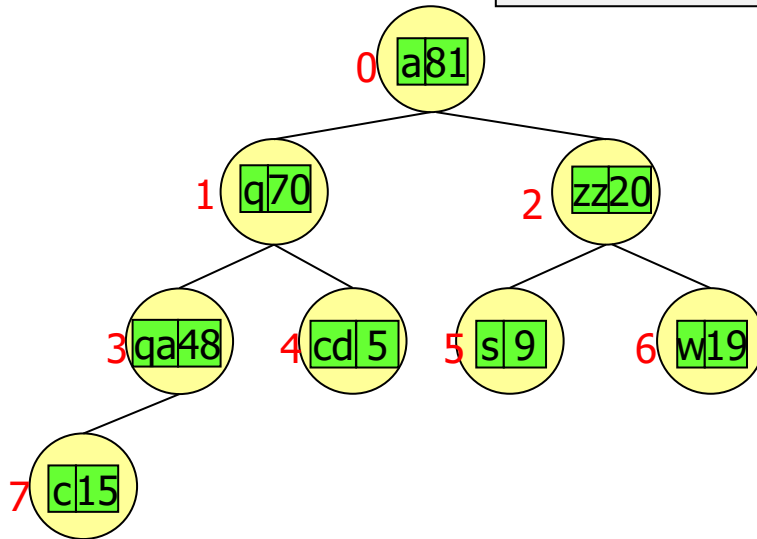
parte da un heap e lo fa risalire mentre per heapify c'è qualcosa che non va

```
void PQinsert (PQ pq, Item val) {  
    int i;  
    i = pq->heapsize++;  
    while((i>=1) &&  
        (KEYcmp(KEYget(pq->A[PARENT(i)]),KEYget(val))==-1)){  
        pq->A[i] = pq->A[PARENT(i)];  
        i = PARENT(i);  
    }  
    pq->A[i] = val;  
    return;  
}
```

Esempio

Inserzione di r 75

pq->A	a	q	zz	qa	cd	s	w	c		
	81	70	20	48	5	9	19	15		
	0	1	2	3	4	5	6	7	8	9
pq->heapsize	8									

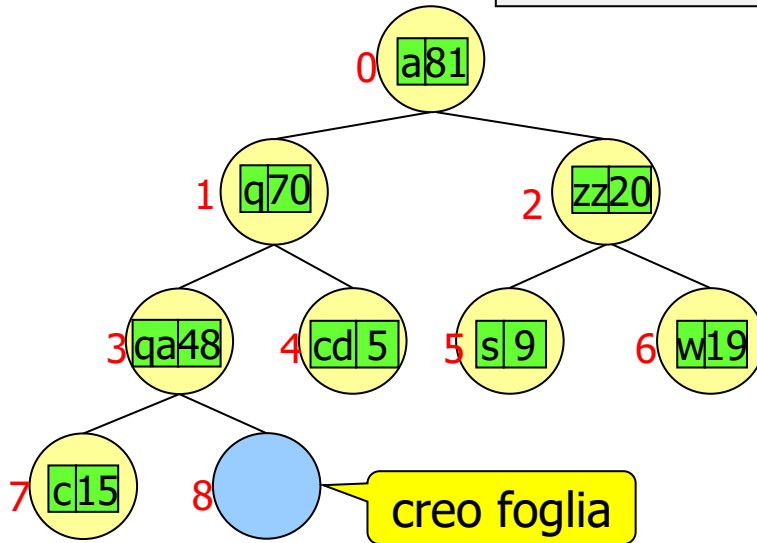


pq->A

a	q	zz	qa	cd	s	w	c		
81	70	20	48	5	9	19	15		

0 1 2 3 4 5 6 7 8 9

pq->heapsize 9



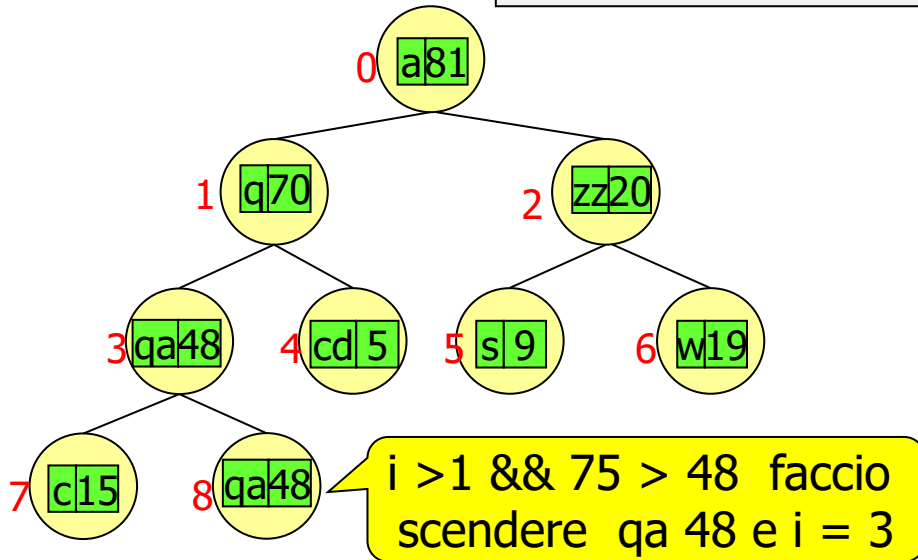
i = 8

pq->A

a	q	zz	qa	cd	s	w	c	qa	
81	70	20	48	5	9	19	15	48	

0 1 2 3 4 5 6 7 8 9

pq->heapsize 9



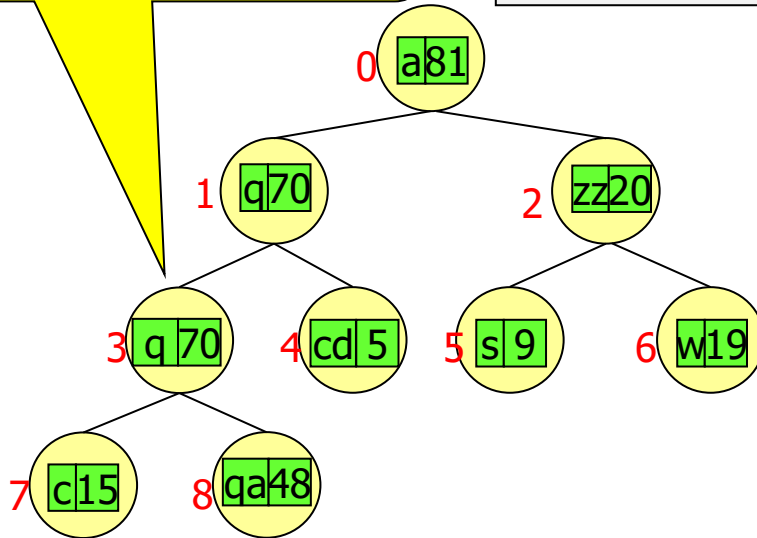
$i > 1 \ \&\& \ 75 > 70$
faccio scendere
q 70 e $i = 1$

pq->A

a	q	zz	q	cd	s	w	c	qa	
81	70	20	70	5	9	19	15	48	

0 1 2 3 4 5 6 7 8 9

pq->heapsize 9



pq->A

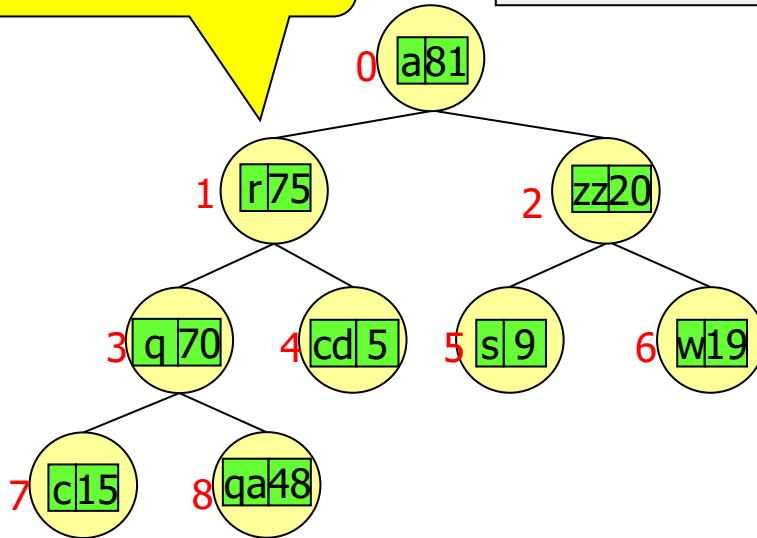
a	r	zz	q	cd	s	w	c	qa	
81	75	20	70	5	9	19	15	48	

0 1 2 3 4 5 6 7 8 9

pq->heapsize

9

$i > 1 \ \&\& \ 75 < 81$
inserisco r 75



Funzione PQextractMax

- Modifica lo heap, estraendone il valore massimo, che è contenuto nella radice:

- scambia la radice con l'ultima delle foglie (quella più a destra nell'ultimo livello)

ottengo un heap che sarebbe giusta tranne la radice

- riduce di 1 della dimensione dello heap

- ripristina le proprietà dello heap mediante applicazione di HEAPIfy.

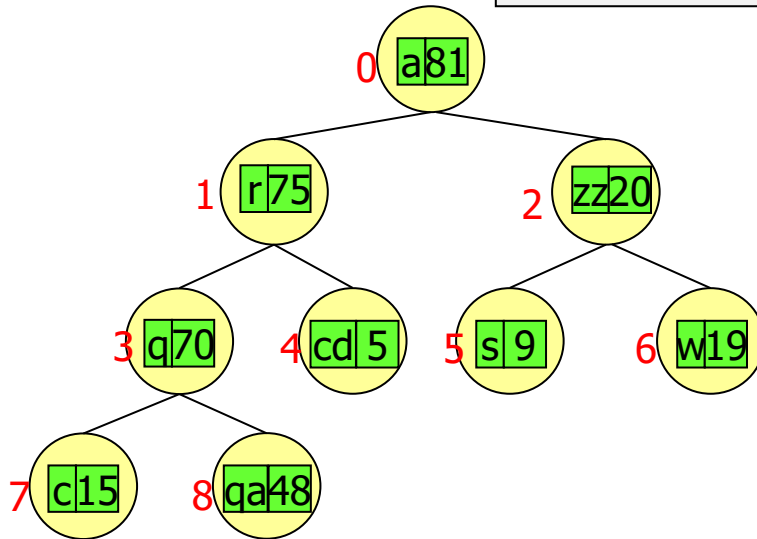
questo per sistemare in maniera lineartmica anzichè lineare

Complessità: $T(n) = O(\lg n)$.

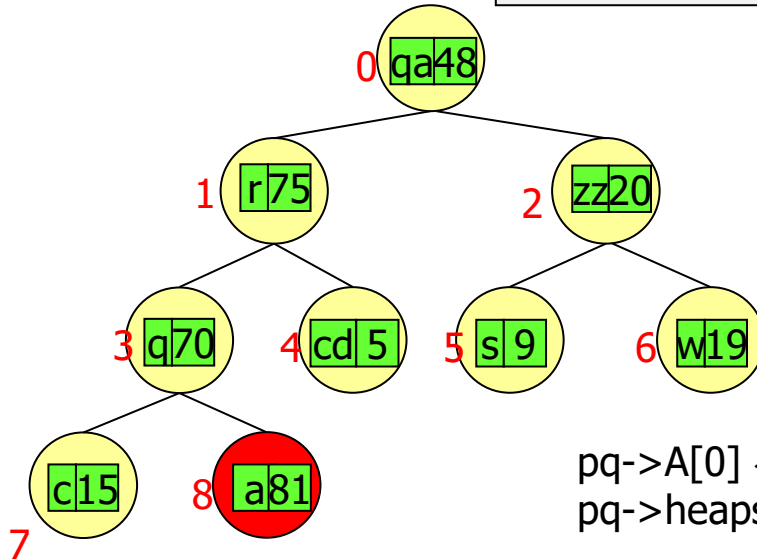

```
Item PQextractMax(PQ pq) {  
    Item val;  
    Swap (pq, 0, pq->heapsize-1);  
    val = pq->A[pq->heapsize-1];  
    pq->heapsize--;  
    HEAPIfy(pq, 0);  
    return val;  
}
```

Esempio

pq->A	a	r	zz	q	cd	s	w	c	qa	
	81	75	20	70	5	9	19	15	48	
	0	1	2	3	4	5	6	7	8	9
pq->heapsize	9									



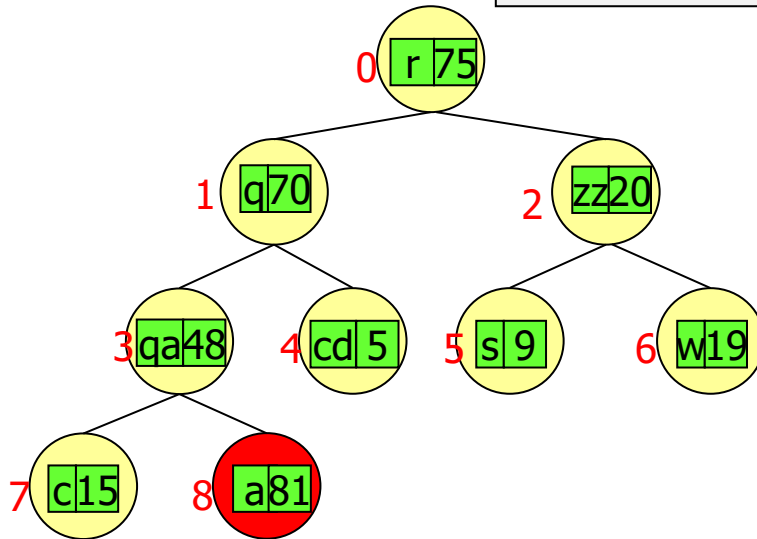
pq->A	qa	r	zz	q	cd	s	w	c	a	
	48	75	20	70	5	9	19	15	81	
	0	1	2	3	4	5	6	7	8	9
pq->heapsize	8									



$\text{pq->A}[0] \leftrightarrow \text{pq->A}[\text{pq->heapsize}-1]$
 pq->heapsize--

HEAPIfy(pq->A, 0)

pq->A	r	q	zz	qa	cd	s	w	c	a	
	75	70	20	48	5	9	19	15	81	
	0	1	2	3	4	5	6	7	8	9
pq->heapsize	8									



Funzione PQchange

- Modifica la priorità di un elemento, la cui **posizione** (indice nello heap) viene calcolata con una scansione di costo lineare
- O risale dalla posizione data fino al più alla radice confrontando la chiave del padre con la chiave modificata, facendo scendere la chiave del padre nel figlio se la chiave modificata è maggiore, altrimenti la inserisce nel nodo corrente
per farlo salire SIMILE a insert
- O applica HEAPIfy a partire dalla posizione data.
per farlo scendere heapify

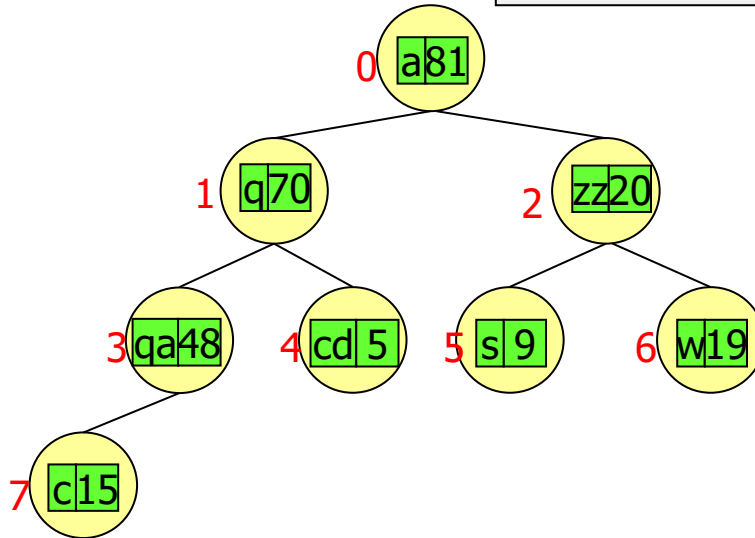
Complessità: $T(n) = O(n) + O(\lg n) = O(n)$.

costo ricerca

diminuisce->scende
aumenta->sale

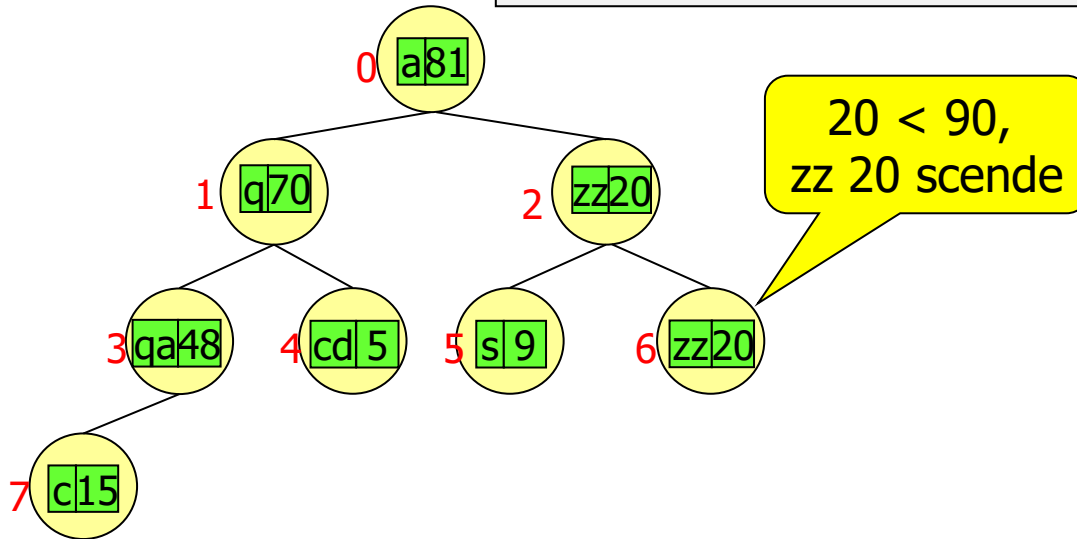
Esempio

pq->A	a	q	zz	qa	cd	s	w	c		
	81	70	20	48	5	9	19	15		
	0	1	2	3	4	5	6	7	8	9
pq->heapsize	8									



Cambio la priorità di w da 19 a 90. L'elemento si trova all'indice 6.

pq->A		a	q	zz	qa	cd	s	zz	c		
		81	70	20	48	5	9	20	15		
		0	1	2	3	4	5	6	7	8	9
pq->heapsize		8									



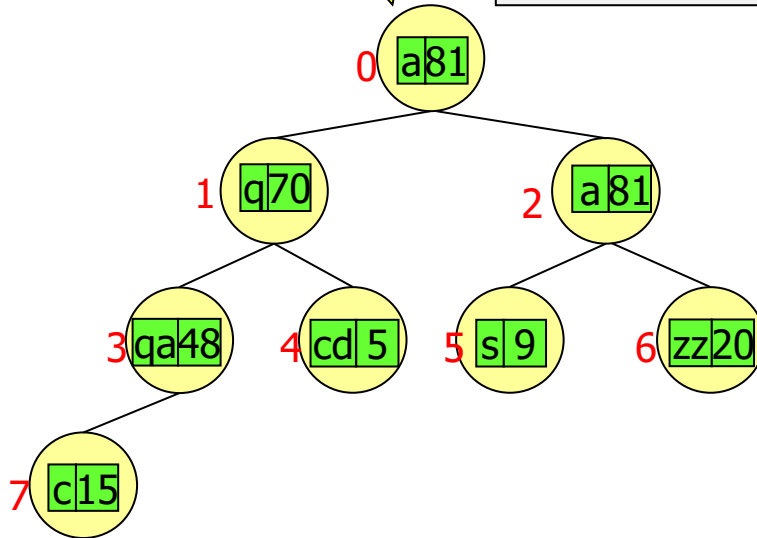
pq->A

a	q	a	qa	cd	s	zz	c		
81	70	81	48	5	9	20	15		

0 1 2 3 4 5 6 7 8 9

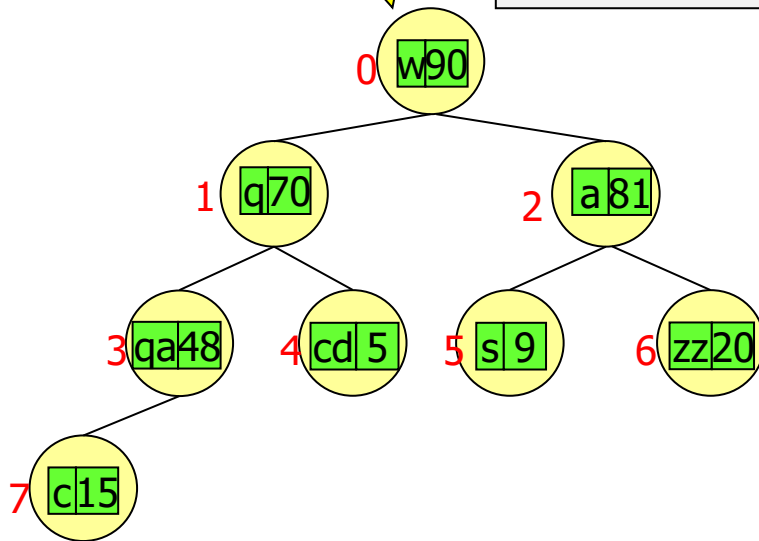
pq->heapsize 8

81 < 90,
a 81 scende



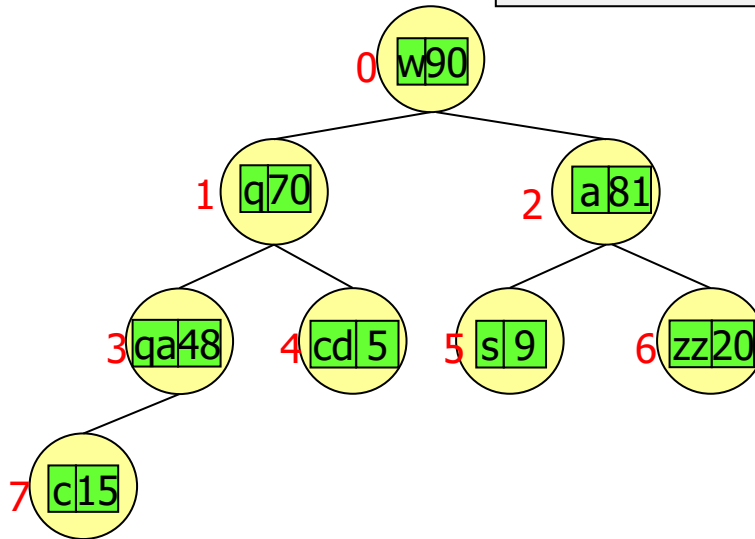
radice:
inserisco w 90

pq->A		w	q	a	qa	cd	s	zz	c		
		90	70	81	48	5	9	20	15		
		0	1	2	3	4	5	6	7	8	9
pq->heapsize		8									



Esempio

pq->A	w	q	a	qa	cd	s	zz	c		
	90	70	81	48	5	9	20	15		
	0	1	2	3	4	5	6	7	8	9
pq->heapsize	8									



Cambio la priorità di q da 70 a 3. L'elemento si trova all'indice 1.

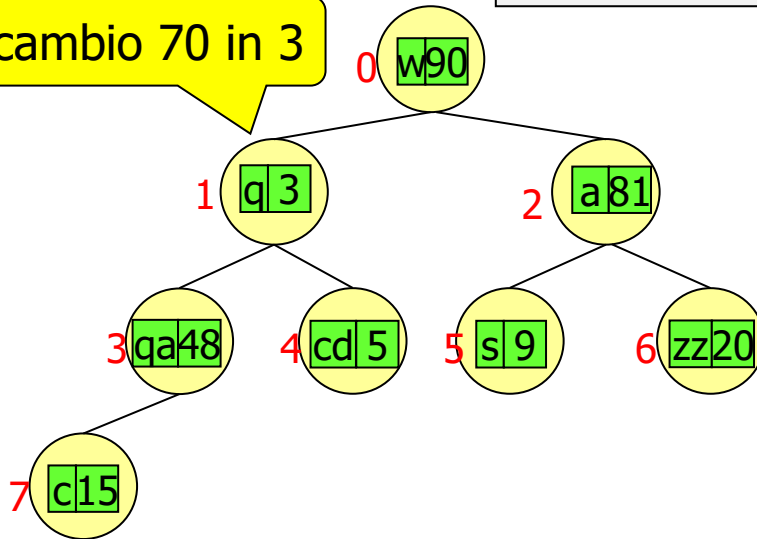
pq->A

w	q	a	qa	cd	s	zz	c		
90	3	81	48	5	9	20	15		

0 1 2 3 4 5 6 7 8 9

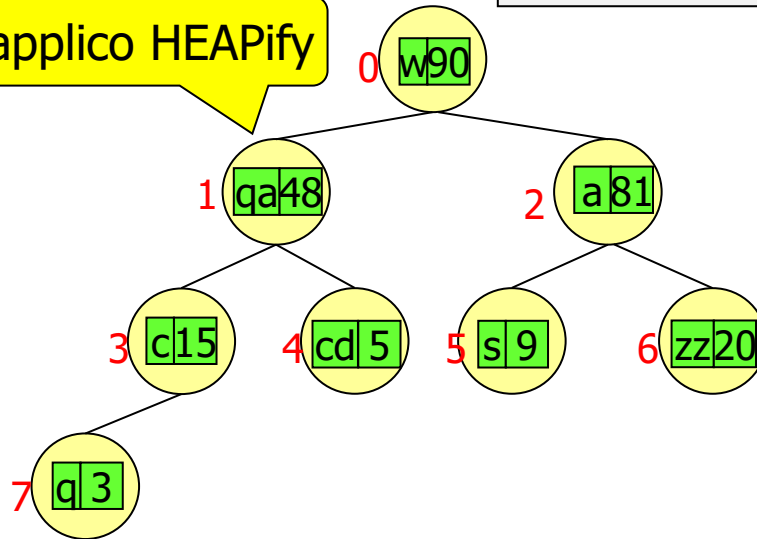
pq->heapsize 8

cambio 70 in 3



pq->A	w	qa	a	c	cd	s	zz	q		
	90	48	81	15	5	9	20	3		
	0	1	2	3	4	5	6	7	8	9
pq->heapsize	8									

applico HEAPIfy



ricerca in base all'item con priorità nuova

prima cerca e poi modifica

IN 3 FASI
CHE CI
SONO
SEMPRE

```
void PQchange (PQ pq, Item val) {
    int i, found = 0, pos;
    for (i = 0; i < pq->heapsize && found == 0; i++)
        if (NAMEcmp(NAMEget(&(pq->A[i])), NAMEget(&val))==0) {
            found = 1;           ricerca lineare
            pos = i;
        }

    if (found==1) {             SIMIL-PQinsert
        while(pos>=1 &&
            PRIOget(pq->A[PARENT(pos)])<PRIOget(val)){   stesso while della PQinsert
            pq->A[pos] = pq->A[PARENT(pos)];             confronta e se ..fa scendere il padre e
            pos = PARENT(pos);
        }
        pq->A[pos] = val;
        HEAPIfy(pq, pos); hepify viene comunque chiamata anche se potrebbe non fare niente
    }
    else
        printf("key not found!\n");
    return;
}
```

È possibile migliorare PQchange $O(n)$?

Occorre fare in modo che NON si debba **cercare** l'item nella coda

- Soluzione: L'Item ricorda/sa «dove» si trova nella coda (gestito dalle operazioni su PQ)

Possibili implementazioni:

- A. In coda si inseriscono solo «riferimenti» ad Item (es. puntatori) item fa riferimento a puntatore
 - l'Item ha un campo pos (posizione in coda prioritaria)
 - Il modulo Item fornisce le operazioni ITEMsetPos ITEMgetPos che permettono di ottenere la posizione di un item con costo $O(1)$
- B. L'Item è un indice (oppure ha come campo valore un indice in un vettore): in pratica è un riferimento a «dove» sono collocate le informazioni. La priorità può essere parte dell'item oppure può essere «affiancata» all'item/indice item riferimento a indice
 - La coda sfrutta internamente la corrispondenza dato-indice, associa a ogni item una casella unica di un vettore
 - E' possibile ottenere la posizione di un Item in coda con tempo $O(1)$
- C. Non è possibile gestire un riferimento ad Item con puntatore o indice, si usa la chiave come riferimento (deve essere univoca, senza possibili duplicati)
 - il modulo PQ usa la chiave dell'item per gestire una corrispondenza chiave-posizione mediante una tabella di simboli efficiente (es. Hash $O(1)$, BST bilanciato $O(\lg(n))$)

facciamo in modo che sia facile cercare un item, e in tab si sia associato la sua posizione in heap
per aver il costo di ricerca lineare o al peggio logaritmico

È possibile migliorare PQchange $O(n)$?

heap deve mantenere traccia di ogni item dove sta

Occorre fare in modo che NON si debba **cercare** l'item nella coda

- Soluzione: L'Item ricorda/sa «dove» si trova nella coda (gestito dalle operazioni su PQ)

Possibili implementazioni:

A. In coda si inseriscono solo «riferimenti» ad Item (es. puntatori)

- l'Item ha un campo pos (posizione in coda prioritaria)
- Il modulo Item fornisce le operazioni ITEMsetPos ITEMgetPos che permettono di ottenere la posizione di un item con costo $O(1)$

B. L'Item è un indice (oppure ha come campo valore un indice in un vettore): in pratica è un riferimento a «dove» sono collocate le informazioni. La priorità può essere parte dell'item oppure può essere «affiancata» all'item/indice

- La coda sfrutta internamente la corrispondenza dato-indice, associa a ogni item una casella unica di un vettore
- E' possibile ottenere la posizione di un Item in coda con tempo $O(1)$

C. Non è possibile gestire un riferimento ad Item con puntatore o indice, si usa la chiave come riferimento (deve essere univoca, senza possibili duplicati)

- il modulo PQ usa la chiave dell'item per gestire una corrispondenza chiave-posizione mediante una tabella di simboli efficiente (es. Hash $O(1)$, BST bilanciato $O(\lg(n))$)

aggiungo un campo per sapere la posizione

Coda prioritaria di indici

Non si inseriscono in coda gli item ma coppie (indice, priorità), quindi si adotta la versione di «chiave affiancata al dato» (la priorità è un parametro aggiuntivo) invece che «chiave parte del dato»

Priority Queue → Queue Position

- Il vettore pq → qp (posizione in coda) serve per implementare Pqchange efficiente, identificando la posizione dell'elemento nello heap con costo $O(1)$ (l'elemento è un indice) senza bisogno di una scansione lineare.

dati fermi e spostiamo gli indici (i riferimenti) tenere aggiornati sia riferimento diretto che inverso

cambia priorità al dato in posizione x senza cambiare locazione del dato dentro la posizione x

priorità AFFIANCA il dato

qp è come il heap ma ha gli indici invertiti con il contenuto della casella

PQ.h

```
typedef struct pqueue *PQ;

PQ      PQinit(int maxN);
void     PQfree(PQ pq);
int      PQempty(PQ pq);
int      PQsize(PQ pq);
void     PQinsert(PQ pq, int index, int prio);
int      PQshowMax(PQ pq);
int      PQextractMax(PQ pq);
void     PQdisplay(PQ pq);
void     PQchange(PQ pq, int index, int prio);
```

PQ.h

```
typedef struct pqueue *PQ;  
  
PQ      PQinit(int maxN);  
void    PQfree(PQ pq);  
int     PQempty(PQ pq);  
int     PQsize(PQ pq);  
void    PQinsert(PQ pq, int index, int prio);  
int     PQshowMax(PQ pq);  
int     PQextractMax(PQ pq);  
void    PQdisplay(PQ pq);  
void    PQchange(PQ pq, int index, int prio);
```

Scompare il tipo Item
Si gestiscono indici e priorità

PQ.c

```
...
typedef struct {int index; int prio} heapItem;
struct pqueue {heapItem *A; int heapsize; int *qp};

PQ PQinit(int maxN) {
    int i;
    PQ pq = malloc(sizeof(*pq));
    pq->A = malloc(maxN*sizeof(heapItem));
    pq->qp = malloc(maxN*sizeof(int));
    for (i=0; i < maxN; i++){
        pq->A[i].index = -1; pq->qp[i] = -1;
    }
    pq->heapsize = 0;
    return pq;
}
```

PQ.c

```
...
typedef struct {int index; int prio} heapItem;
struct pqueue {heapItem *A; int heapsize; int *qp};

PQ PQinit(int maxN) {
    int i;
    PQ pq = malloc(sizeof(*pq));
    pq->A = malloc(maxN*sizeof(heapItem));
    pq->qp = malloc(maxN*sizeof(int));
    for (i=0; i < maxN; i++){
        pq->A[i].index = -1; pq->qp[i] = -1;
    }
    pq->heapsize = 0;
    return pq;
}
```

Item interno al modulo PQ
per gestire la coppia
(indice,priorità)

PQ.c

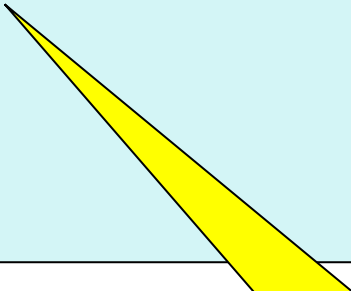
```
...
typedef struct {int index; int prio} heapItem;
struct pqueue {heapItem *A; int heapsize; int *qp};

PQ PQinit(int maxN) {
    int i;
    PQ pq = malloc(sizeof(*pq));
    pq->A = malloc(maxN*sizeof(heapItem));
    pq->qp = malloc(maxN*sizeof(int));
    for (i=0; i < maxN; i++){
        pq->A[i].index = -1; pq->qp[i] = -1;
    }
    pq->heapsize = 0;
    return pq;
}
```

Si assume che maxN sia,
oltre che il massimo numero
di dati in coda,
il limite superiore agli indici

```
void PQfree(PQ pq) {  
    free(pq->qp);  
    free(pq->A);  
    free(pq);  
}  
  
int PQempty(PQ pq){  
    return pq->heapsize == 0;  
}  
  
int PQsize(PQ pq) {  
    return pq->heapsize;  
}
```

```
void PQinsert (PQ pq, int index, int prio){
    int i, j;
    i=pq->heapsize++;
    while((i>=1) &&
        (pq->A[PARENT(i)].prio)<prio)){
        pq->A[i] = pq->A[PARENT(i)];
        pq->qp[pq->A[i].index] = i;
        i = PARENT(i);
    }
    pq->A[i].index = index;
    pq->A[i].prio = prio;
    pq->qp[index] = i;
}
```



aggiorno pq->qp

```
static void Swap(PQ pq, int pos1, int pos2){  
    heapItem temp;  
    int index1, index2;  
    temp = pq->A[pos1];  
    pq->A[pos1] = pq->A[pos2];  
    pq->A[pos2] = temp;  
    // update correspondence index-pos  
    index1 = pq->A[pos1].index;  
    index2 = pq->A[pos2].index;  
    pq->qp[index1] = pos1;  
    pq->qp[index2] = pos2;  
}
```



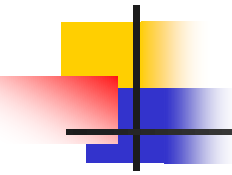
```
static void HEAPIfy(PQ pq, int i) {
    int l, r, largest;
    l = LEFT(i);
    r = RIGHT(i);
    if (l < pq->heapsize && (pq->A[l].prio > pq->A[i].prio))
        largest = l;
    else
        largest = i;
    if (r < pq->heapsize && (pq->A[r].prio > pq->A[largest].prio))
        largest = r;
    if (largest != i) {
        Swap(pq, i, largest);
        HEAPIfy(pq, largest);
    }
}
```

```
int PQextractMax(PQ pq) {  
    int res;  
    int j=0;  
  
    Swap (pq, 0, pq->heapsize-1);  
    res = pq->A[pq->heapsize-1].index;    differenza  
    pq->qp[res]=-1;  
    pq->heapsize--;  
    pq->A[pq->heapsize].index=-1; // redundant  
    HEAPIfy(pq, 0);  
    return res;  
}
```

```
void PQchange (PQ pq, int index, int prio) {
    int pos = pq->qp[index];
    heapItem temp = pq->A[pos];
    temp.prio = prio; // new prio

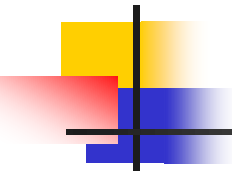
    while ((pos>=1) && (pq->A[PARENT(pos)].index < prio) {
        pq->A[pos] = pq->A[PARENT(pos)];
        pq->qp[pq->A[pos].index] = pos;
        pos = PARENT(pos);
    }
    pq->A[pos] = temp;
    pq->qp[index] = pos;

    HEAPIfy(pq, pos);
}
```



Riferimenti

- Heap:
 - Cormen 7.2, 7.3
 - Sedgewick 9.2, 9.3
- Heapsort:
 - Cormen 7.4
 - Sedgewick 9.4
- Code a priorità:
 - Cormen 7.5
 - Sedgewick 9.1, 9.6



Esercizi di teoria

- 7. Code a priorità e heap
 - 7.1 Heap
 - 7.2 Heap Sort
 - 7.3 Code a priorità

